FACULDADE DE CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE D COIMBRA

departamento de engenharia informática

Systems and Database Development

Library Management REST API

SGD Project Report

Clara Pérez Alonso
Gabriel Furtado

Contents

1 Introduction

The main objective of this project was to build a RESTful API for a library management system using PostgreSQL and Python. The core idea is to manage the library's daily operations: users borrow books, return them, leave reviews, and manage the collection according to their role.

As we delved deeper, things became more interesting and, at the same time, more complex. Transactions, concurrent, consistency, secure password storage—all of this quickly accumulated. Fixing those issues was probably the hardest part.

The application follows a client-server architecture where the REST API acts as the communication layer between the user and the PostgreSQL database. Flask was used as the backend framework because of its simplicity and flexibility, while PostgreSQL provided the transactional and relational functionalities required by the project specifications. Additionally, we also used Postman a lot during development and testing to validate the behavior of all endpoints.

Another important challenge was generating synthetic data. If you only test with a little data, you miss problems that show up when the database is full. That helped us catch issues we would have missed otherwise.

Throughout the development process, several design decisions had to be made regarding authentication, relational modeling, transaction management, and validation strategies. Some solutions worked immediately, while others required multiple iterations and adjustments until they were stable. In many ways, this was a trial-and-error process.

In the end, the system successfully implements all required endpoints and satisfies the project specifications. It handles all 16 endpoints, follows the rules from the spec, and after a few tries, runs without issues.

2 System Overview

The system developed for this project is a library management platform accessible through a REST API. Three different types of users interact with the application: administrators, librarians, and readers, each one with specific permissions and responsibilities inside the system.

- Administrators are responsible for managing the book collection. Through the API implemented in the file `api.py`, they can add new books, update the number of available copies, and register authors in the database.
- Librarians focus more on monitoring and reporting tasks. They are able to consult the complete list of active loans, search books by title or ISBN, verify who currently has a specific book borrowed, and generate reports such as the most borrowed books or the most active readers over a given period of time.
- Readers represent the regular users of the library: they browse books by genre, borrow and return titles, and submit reviews containing both a rating and a written comment.

The relational schema defined in the file `create_db.sql` was designed to reflect realistic library constraints and business rules. For instance, a reader cannot borrow more than five books simultaneously, reviews can only be submitted for books previously borrowed by that reader, and loan records are never deleted from the system. Loan records are never deleted, returns are recorded by updating a date field, preserving the full borrowing history.

One of the most delicate parts of the implementation was concurrency management during the borrowing process. Without proper transaction control, two simultaneous requests could both succeed when only one copy is available, creating inconsistent data. This was handled through PostgreSQL transaction locking,

explained in detail in Section 5.

Overall, the system combines relational database management, transaction control, authentication, and reporting functionalities inside a single REST-based architecture.

3 Data Modeling

3.1 Conceptual Model (ER Diagram)

Designing the data model was one of the first real challenges of the project. Before starting the code, we needed to understand how users, books, loans, reviews, authors, and genres fit together.

The central entity is USER, which stores the common information of all registered users: a unique identifier, username, email address, and password. From there, three specialization roles were defined: READERS, LIBRARIAN, and ADMINISTRATOR, using a complete specialization hierarchy.

This means that each user in the system belongs to only one of these roles, and the role determines the operations they can perform.

Several types of relationships were defined in the conceptual model depending on how the entities interact with each other. Minimum and maximum cardinalities were also considered during the design process.

The relationship between READERS and LOAN through BORROWS is a one-to-many (1:N) relationship. A reader may have zero or many loans over time, while each loan belongs to exactly one reader. This means the participation of LOAN is mandatory, while the participation of READERS is optional.

The relationship between BOOK and LOAN through IS_LOANED is also one-to-many (1:N). A book may appear in zero or many loan records, while each loan is associated with exactly one book. Loans cannot exist without a corresponding book.

The relationship between READERS and REVIEW through WRITES follows the same structure. One reader may write zero or many reviews, while each review belongs to exactly one reader.

Similarly, the relationship between REVIEW and BOOK through REVIEWS is one-to-many (1:N). A book may receive zero or many reviews, while each review references exactly one book.

The relationship between ADMINISTRATOR and BOOK through MANAGES allows one administrator to manage zero or many books, while a book may optionally be associated with one administrator. This optional participation exists because some books may remain stored even if the administrator account is removed later.

The relationships between BOOK and GENRE, and between BOOK and AUTHOR, are many-to-many (N:M) relationships. A book may belong to multiple genres and may have multiple authors, while genres and authors may also be associated with multiple books. These relationships were later implemented using the intermediate tables book_genre and book_author.

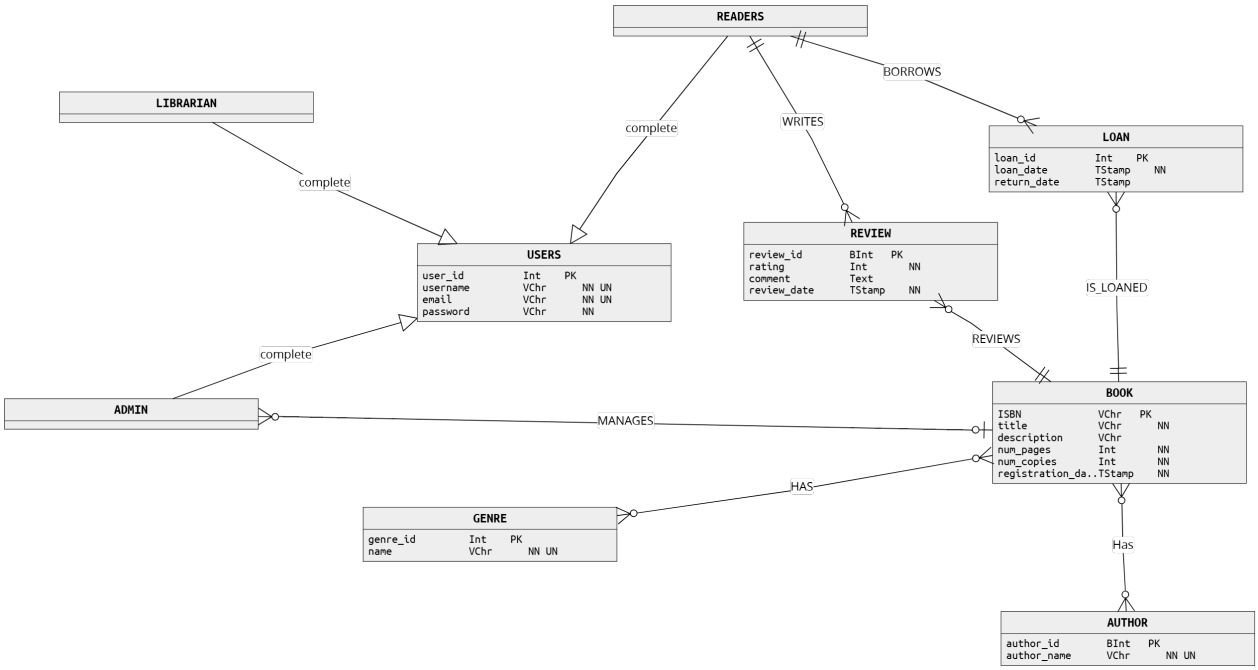

Figure 1: Conceptual ER Diagram (ONDA)

3.2 Relational Schema

Translating the conceptual model into a relational schema meant making more concrete decisions about how each entity and relationship would actually be represented in PostgreSQL.

The specialization hierarchy was implemented using separate tables for each role, administrator, librarian, and readers, each containing only a foreign key pointing back to `users`. This avoids duplicating user information while still allowing the system to identify the role of each user efficiently.

The two many-to-many relationships, between books and authors, and between books and genres, were resolved using intermediate tables: `book_author` and `book_genre`. These tables store pairs of foreign keys and use composite primary keys, which also prevents duplicate associations from being inserted accidentally.

The full database schema implemented in `create_db.sql` is composed of the following tables:

- `users(user_id, username, email, password)`
- `administrator(users_user_id)`
- `librarian(users_user_id)`
- `readers(users_user_id)`
- `book(isbn, title, description, num_pages, num_copies, registration_date, administrator_users_user_id)`
- `author(author_id, author_name)`
- `genre(genre_id, name)`
- `book_author(book_isbn, author_author_id)`
- `book_genre(book_isbn, genre_genre_id)`
- `loan(loan_id, loan_date, return_date, readers_users_user_id, book_isbn)`

- review(review_id, rating, comment, review_date, readers_users_user_id, book_isbn)

Two partial indexes were also created to improve the performance of the most frequent queries involving active loans:

```
CREATE INDEX idx_loan_active ON loan (book_isbn)
WHERE return_date IS NULL;

CREATE INDEX idx_user_loans ON loan (readers_users_user_id)
WHERE return_date IS NULL;
```

Since most loan-related operations filter by return_date IS NULL, these indexes avoid unnecessary full table scans and make borrowing validations significantly faster, especially when working with larger datasets.

3.3 Integrity Constraints and Relationships

Beyond the structure of the tables themselves, a considerable amount of attention was given to integrity constraints and validation rules, both at the database level and inside the API logic implemented in `api.py`.

At the database level, `create_db.sql` defines several CHECK constraints directly in PostgreSQL:

- `num_pages > 0`: a book must contain at least one page.
- `num_copies >= 0`:the number of copies may reach zero but can never become negative.
- `rating BETWEEN 1 AND 5`: reviews outside this range are rejected directly by the database.
- `return_date IS NULL OR return_date >= loan_date`: a return date cannot be earlier than the loan date.
- `UNIQUE(readers_users_user_id, book_isbn)` on `review`: ensuring that each reader can only have one review per book.

Foreign key constraints with ON DELETE CASCADE were used for the role tables, meaning that deleting a user also removes the corresponding entry from readers, librarian, or administrator. On the other hand, the foreign key between book and administrator uses ON DELETE SET NULL, so that books remain stored in the database even if the administrator who created them is later removed.

On top of the database-level constraints, additional business rules are enforced through the API logic:

- A reader cannot have more than five active loans simultaneously.
- A reader cannot borrow a book they already have on an active loan.
- A review can only be submitted for a book previously borrowed by that reader.
- If a reader submits a second review for the same book, the existing review is updated instead of creating a duplicate.

In practice, combining database-level constraints with API-side validation made the system significantly more reliable.

4 API Design and Implementation

4.1 REST Architecture

The API follows a REST architecture, meaning that each endpoint represents either a resource or a specific operation, while the interaction happens through standard HTTP methods such as GET, POST, and PUT. All communication between the client and the API uses JSON objects for both requests and responses, following the requirements defined in the project specification.

In order to follow REST principles, different HTTP methods were used depending on the type of operation being performed.

- GET requests were used for operations that only retrieve information from the database without modifying any data, such as searching books, viewing reviews, or generating reports.
- POST requests were mainly used for operations that create new records or trigger actions inside the system, such as registering users, borrowing books, returning books, submitting reviews, or adding authors and books.
- PUT requests were used for update operations where existing data needed to be modified. In this project, the update_copies endpoint uses PUT because it updates the number of copies associated with an already existing book.

Every response returned by the API follows the same structure:

```
{   "status": status_code,
    "errors": ...,
    "results": ...}
```

The field indicates whether the operation was successful or not. The results field contains the requested data or the result of the operation when the request succeeds, while the errors field contains an explanatory error message whenever the request fails.

Status code 200 indicates a successful operation. Status code 400 is used whenever the request itself is invalid, for example, if required fields are missing, credentials are incorrect, or the user does not have permission to execute the operation. Finally, 500 represents unexpected internal server or database errors.

The API was implemented in the file `api.py` using Flask. One important detail is that Flask runs in threaded mode (threaded=True), meaning multiple requests may be processed concurrently. This became especially relevant for the borrowing process, where simultaneous requests for the same book needed proper transaction control to avoid inconsistencies.

4.2 Endpoint Organization

The API endpoints were implemented according to the business operations required by the project specification. Although permissions depend on the user role, the endpoints are mainly organized by functionality, including book management, loan operations, reviews, reports, and validation tests.

Some endpoints are restricted to administrators, others can be accessed by librarians and administrators, while some operations are available to all authenticated users.

Administrator-only operations:

- `POST /sgdproj/book` — add a new book
- `PUT /sgdproj/update_copies` — modify the number of copies of a book
- `POST /sgdproj/author` — register a new author

The `/author` endpoint was not part of the original mandatory specification, but it was added during development to simplify the insertion and management of authors before linking them to books.

Administrator and librarian operations:

- `GET /sgdproj/list_books_by_genre`: list all books in a genre
- `GET /sgdproj/find_book_by_name`: search books by title
- `GET /sgdproj/find_book_by_isbn`: find a book by ISBN
- `GET /sgdproj/loaned_books`: list all currently active loans
- `GET /sgdproj/loaned_book`: loan history of a specific book
- `GET /sgdproj/TopLoanedBooks/{N}`: top N most borrowed books
- `GET /sgdproj/TopLoaners/{N}`: top N most active readers
- `GET /sgdproj/report/TopLoanedGenres12Months/{N}`: top N borrowed genres during the last 12 months

Authenticated user operations:

- `POST /sgdproj/user`: register a new user
- `GET /sgdproj/available_books_by_genre` : browse available books
- `GET /sgdproj/check_book`: view reviews for a book
- `POST /sgdproj/borrow_book`: borrow a book
- `POST /sgdproj/report/submit_review`: submit or update a review
- `POST /sgdproj/return_book`: return a book

Before executing any operation, each endpoint validates whether all required fields are present in the request body. If something is missing, the API immediately returns a 400 response without interacting with the database. Only after validation does the system proceed with authentication and permission checks.

4.3 Authentication and Authorization

Authentication and authorization were implemented directly inside the API in order to control access to the different operations available in the system. Since the project specification did not require sessions or token-based authentication, every protected request includes the username and password inside the request body.

This means that each request is authenticated independently, making every operation completely self-contained. Although this approach is simpler than using JWT tokens or session management, it was sufficient for the requirements of the project and easier to maintain during development.

The authentication process starts by validating whether the username exists in the `users` table. If the user exists, the password provided in the request is compared against the bcrypt hash stored in PostgreSQL using `bcrypt.checkpw()`. If either the username or password is invalid, the request is rejected immediately with a 400 response.

Passwords were never stored in plain text, neither in the database nor in the CSV files generated by `generate_csv.py`. Before insertion, all passwords were hashed using bcrypt together with randomly generated salts. As a result, even users with identical passwords end up with completely different stored hashes.

Bcrypt was intentionally chosen over simpler hashing algorithms such as MD5 or SHA because it is slower by design and therefore significantly more resistant to brute-force attacks. Even though this slightly increases authentication time, it greatly improves credential security.

After authentication succeeds, the API performs authorization checks depending on the endpoint being accessed. Different operations require different user roles:

- Administrator-only endpoints are restricted to users stored in the `administrator` table.
- Reporting and monitoring operations are shared between administrators and librarians.
- Borrowing, returning books, and submitting reviews require the user to belong to the `readers` role.

Role verification is performed through additional SQL queries after authentication. If the user does not have the required permissions, the API returns controlled error responses such as:

```
{
    "status": 400,
    "errors": "User not authorized"
}
```

This separation between authentication and authorization simplified the implementation and made permission handling easier to debug during testing.

5 Database Execution and Transactions

5.1 Concurrency Handling

Out of all the implemented endpoints, the borrow operation was the one that required the most careful thought regarding concurrency. The problem itself is relatively simple to describe, but surprisingly easy to implement incorrectly: what happens if two readers attempt to borrow the last available copy of the same book at nearly the same time?

Without proper synchronization, both requests could read the same availability state simultaneously, both could conclude that one copy is still available, and both could insert a new loan record. The result would be an inconsistent database state with more active loans than actual copies.

To prevent this situation, the borrow endpoint implemented in `api.py` uses PostgreSQL row-level locking through SELECT FOR UPDATE. Before validating availability, the transaction locks both the target book row and the active loan rows associated with that book:

```
cur.execute(
    """
        SELECT num_copies
        FROM book
        WHERE isbn = %s
        FOR UPDATE
    """,
    (isbn,)
)
```

The system also avoids storing available copies as a persistent database field. Instead, availability is calculated dynamically by subtracting active loans from the total number of copies stored in the book table:

```
SELECT COUNT(*)
FROM loan
WHERE book_isbn = %s
AND return_date IS NULL
```

This approach avoids synchronization problems that would appear if available copies were stored separately and updated manually after every borrow or return operation. By calculating the value directly from the loan records, the system reduces the risk of inconsistencies.

5.2 Transaction Management

All database operations inside `api.py` follow an explicit transaction management strategy using `try/except/finally` blocks combined with manual commits and rollbacks.

Since psycopg2 does not enable autocommit by default, database changes are only persisted when `conn.commit()` is called explicitly. If any operation fails before that point, `conn.rollback()` restores the previous database state:

```
try:
    cur.execute(...)
    conn.commit()

    return jsonify({
        'status': 200,
        'results': ...
    })

except (Exception, psycopg2.DatabaseError) as e:
    conn.rollback()

    return jsonify({
        'status': 500,
        'errors': str(e)
    })

finally:
    conn.close()
```

This same transaction pattern is repeated throughout most endpoints that modify data inside the database. The finally block guarantees that connections are always closed properly, regardless of whether the transaction succeeds or fails.

The borrow operation is the most complex transaction implemented in the project because it combines multiple validations and locking operations before inserting the final loan record:

1. Authenticate the user.
2. Verify the user belongs to the readers role.
3. Lock the target book row using FOR UPDATE.
4. Check that the reader has fewer than five active loans.
5. Verify the reader has not already borrowed the same book.
6. Lock and count active loans for the selected book.
7. Insert the new loan record if copies are available.

8. Commit the transaction.

If any validation fails during this sequence, the transaction is rolled back immediately and no changes are written to the database. This all-or-nothing behavior is essential for keeping the system consistent under concurrent access and unexpected errors.

Another important detail is that the API frequently performs explicit rollbacks even for logical validation errors, such as duplicate borrows or invalid review submissions. Although some of these situations do not necessarily raise database exceptions, rolling back explicitly keeps transaction handling consistent throughout the entire application.

5.3 Error Handling

Error handling follows a relatively simple but consistent strategy throughout the entire API. In practice, most errors can be grouped into three main categories: request validation errors, business logic validation errors, and unexpected database errors.

The first layer is **request validation**. Before performing any SQL query, endpoints verify that all required fields are present in the request body:

```
required_fields = ['username', 'password', 'ISBN']

for field in required_fields:
    if field not in payload:
        return jsonify({
            'status': 400,
            'errors': f'Missing required field: {field}'
        })
```

If required information is missing, the API immediately returns a 400 response without interacting with the database. This keeps validation fast and avoids unnecessary database operations.

The second layer is business logic validation. Once the request structure is valid and the user is authenticated, the API checks whether the requested operation actually makes sense according to the business rules defined for the system.
Examples include:

- verifying that a book exists,
- checking whether copies are available,
- ensuring that the user has the correct role,
- preventing duplicate active loans,
- or verifying that a review is only submitted for a previously borrowed book.

These situations also return status code 400 with descriptive messages explaining the problem:

```
if active_loans >= 5:
    return jsonify({
        'status': 400,
        'errors': 'User already has 5 borrowed books'
    })
```

The final layer handles unexpected database or server errors. These exceptions are captured by generic exception handlers and returned as status code 500 responses:

```
except (Exception, psycopg2.DatabaseError) as e:
    conn.rollback()

    return jsonify({
        'status': 500,
        'errors': str(e)
    })
```

One particularly important case involves duplicate entries during user registration or book insertion. PostgreSQL raises a `UniqueViolation` exception whenever a UNIQUE constraint is violated. Instead of allowing this exception to propagate as a generic server error, the API catches it explicitly and returns a much clearer client-side message:

```
except psycopg2.errors.UniqueViolation:
    conn.rollback()

    return jsonify({
        'status': 400,
        'errors': 'Username or email already exists'
    })
```

The API also includes a logging system configured inside `api.py`. During execution, important events such as requests, validation errors, warnings, and unexpected exceptions are written into the file `logs/log_file.log`.

This became especially useful during development because it allowed us to trace endpoint execution step by step and identify where transactions or validations were failing. In several cases, the log file helped detect issues that were difficult to reproduce manually through Postman alone.

Overall, this layered approach to validation and error handling helped keep the API stable and predictable even when invalid requests, concurrent access, or unexpected failures occurred. Instead of failing silently or leaving inconsistent data behind, the system always attempts to return a controlled response while preserving database integrity.

6 Synthetic Data Generation

Generating realistic data for the library system ended up being more important than we initially expected. Testing with only a small number of manually inserted records was enough to verify that the endpoints worked, but it did not expose many of the situations that appear once the database becomes larger and more active.

Problems such as readers reaching the maximum loan limit, books running out of available copies, duplicated reviews, or some users borrowing significantly more books than others only became visible after working with a larger dataset.

To properly test the API under more realistic conditions, we developed the script `generate_csv.py`, which uses the Python `Faker` library to automatically generate all CSV files required to populate the database. The script produces datasets for users, readers, librarians, administrators, books, authors, genres, loans, reviews, and the intermediate tables `book_author` and `book_genre`.

The final generated dataset contains:

- 115 total users (110 readers, 4 librarians, and 1 administrator)
- 1000 books distributed across 10 genres
- 200 authors
- 7000 loans

- 450 reviews

Even though the dataset is still relatively small compared to a real production system, it was already large enough to expose several issues that never appeared during early testing with manually inserted records.

Books and Authors

Book titles were generated using small randomized templates in order to create variety without requiring a manually curated dataset. Some examples include:

- *The* {*a*} *of* {*b*}
- *Beyond the* {*a*}
- {*a*} *Chronicles*
- *Return to the* {*a*}

The goal was not necessarily to generate perfect literary titles, but rather to create data that looked believable enough during searches, reports, and API responses.

Authors were generated using localized Faker providers so that the dataset would contain names from different nationalities, including Portuguese, Spanish, French, Italian, and English backgrounds.

Each book was associated with between one and three authors, and between one and two genres. Additionally, more than 80% of books belong exclusively to a single genre.

Realistic Borrowing Behavior

One of the main goals during data generation was avoiding a perfectly uniform distribution of activity. In practice, real libraries do not behave uniformly: some readers borrow books constantly while others barely interact with the system, and some books become significantly more popular than others.

To simulate this behavior, the generation script randomly selected a subset of users as top readers. These users had a much higher probability of appearing in generated loan records. Similarly, a subset of books was treated as popular books, increasing the likelihood that they would be borrowed repeatedly.

This generated a more natural distribution of borrowing activity across both users and books, making the statistical reports more representative of a real library environment. Without this weighted generation process, endpoints such as /TopLoanedBooks/{N} and /TopLoaners/{N} would have returned very similar results for most entries, producing unrealistic statistics.

Business Rules During Generation

Several business constraints implemented later in the API were already enforced directly during dataset generation in order to avoid loading inconsistent data into PostgreSQL from the start.

The script guarantees that:

- no reader has more than five active loans at the same time,
- no book exceeds its available copy limit,
- duplicate active loans for the same reader-book pair cannot exist,
- reviews are only generated for books previously borrowed by the corresponding reader,
- duplicate reviews are avoided,

- and ratings follow weighted probabilities where higher ratings appear more frequently than lower ones.

This last detail was intentionally added because completely uniform ratings looked unrealistic during testing. Most users tend to review books positively unless they had a particularly bad experience.

Active loans were represented using NULL return dates, while returned books received a randomly generated return date after the original loan date. All generated dates were kept in the past relative to the current system date in order to avoid temporal inconsistencies during testing.

Passwords were also generated automatically with randomized complexity and then hashed using bcrypt before being written to the CSV files.

Loading the Data

All generated CSV files are loaded into PostgreSQL using the script `load_data.py`. Before inserting any records, the script executes `create_db.sql`, which recreates the entire database schema, including tables, constraints, foreign keys, and indexes.

After that, the CSV files are inserted in a carefully controlled order in order to respect foreign key dependencies. For example, users must be inserted before readers or librarians, books must exist before loans can reference them, and loans must exist before reviews are generated.

This loading sequence became especially important once the dataset grew larger, since even a small ordering mistake could cause hundreds of foreign key violations during insertion.

7 Testing and Validation

7.1 Postman Collection

All API endpoints were tested using Postman throughout the development process. In practice, Postman became one of the most useful tools in the entire project, especially once the system started growing and the number of endpoints, validation rules, and possible error cases increased.

The complete testing collection is included in the project submission as: `LIBRARY_API_-_SGD_Project.postman_collectio`

The collection contains the sixteen mandatory endpoints defined in the specification, the additional endpoint created for adding authors, and several requests intentionally designed to trigger errors and validate permission handling, transaction rollback, and input validation.

During testing, different accounts were used depending on the role required by the endpoint:

- **Administrator account:** `danielpotter` / `_4l!LZlsKm`
- **Reader account:** `garrett.brewer` / `7b8SqN+l^Z`
- **Reader account for error testing:** `ksmith` / `u3B*aM+y^l`

7.2 Endpoint Validation

The Postman collection includes requests for all implemented operations:

- Registering users
- Adding authors
- Adding books
- Updating the number of copies

- Listing books by genre
- Searching books by title or ISBN
- Viewing available books and reviews
- Viewing active loans and loan history
- Borrowing and returning books
- Submitting and updating reviews
- Generating reports such as:
  - Top Loaned Books
  - Top Loaners
  - Top Loaned Genres during the last 12 months
- Executing validation and error-case requests

Each request validates both the returned JSON structure and the correct execution of the underlying business logic.

The tests also verified that PostgreSQL constraints and API-side validation behaved consistently together. This included checking duplicate prevention, role restrictions, maximum active loan limits, invalid review submissions, and availability calculations during concurrent borrow operations.

For example, the /borrow_book endpoint was tested under multiple conditions:

- successful borrow,
- borrowing a non-existing book,
- borrowing without reader permissions,
- borrowing a book already borrowed,
- exceeding the 5-book loan limit,
- and attempting concurrent borrows when copies are low.

Similarly, the review endpoint was tested not only for normal review insertion, but also for review updates, invalid ratings, and attempts to review books never borrowed by the reader.

The collection also helped verify that role-based authorization was working correctly. For instance, reader accounts attempting administrator-only operations consistently returned 400 errors with descriptive messages such as:

```
{"errors": "User not authorized", "status": 400}
```

7.3 Example Requests and Responses

The following examples illustrate some representative requests included in the Postman collection.

Register User (POST)

```
POST http://localhost:8080/sgdproj/user

{
    "username": "newreader1",
    "email": "newreader1@email.com",
    "password": "password123",
    "role": "reader"
}
```

Expected response:

```
{
    "status": 200,
    "results": user_id
}
```

Add Book (POST)

```
POST http://localhost:8080/sgdproj/book

{
    "username": "danielpotter",
    "password": "_4l!LZlsKm",
    "ISBN": "978-0-00-111222-3",
    "book_name": "Test Book",
    "authors": ["Christopher Davis"],
    "book_description": "A test book description",
    "pages": 300,
    "copies": 5,
    "genres": ["Fantasy"]
}
```

Expected response:

```
{
    "status": 200,
    "results": "978-0-00-111222-3"
}
```

Borrow Book (POST)

```
POST http://localhost:8080/sgdproj/borrow_book

{
    "username": "garrett.brewer",
    "password": "7b8SqN+l^Z",
    "ISBN": "978-0-00-111222-3"
}
```

Expected response:

```
{
    "status": 200,
    "results": "978-0-00-111222-3"
}
```

Submit Review (POST)

```
POST http://localhost:8080/sgdproj/report/submit_review
```

```
{
    "username": "amberowens",
    "password": "QMA1Ekr4(D",
    "ISBN": "979-0-617-697935-1",
    "rating": 5,
    "comment": "Amazing book."
}
```

Expected response:

```
{
    "status": 200,
    "results": "979-0-617-697935-1"
}
```

Top Loaned Books Report (GET)

```
GET http://localhost:8080/sgdproj/TopLoanedBooks/5
```

```
{
    "username": "danielpotter",
    "password": "_4l!LZlsKm"
}
```

Expected response:

```
{
    "status": 200,
    "results": [
        {
            "ISBN": "...",
            "book_name": "...",
            "loaned_times": 71,
            "rank": 1
        }
    ]
}
```

Error Validation

Besides normal requests, the collection also contains several intentional error cases.

Some representative examples include:

- borrowing a non-existing book,
- returning a book without an active loan,
- registering duplicated users,
- submitting reviews for books never borrowed,
- and attempting administrator-only operations with a normal reader account.

For example, attempting to borrow a non-existing book returns:

```
{
    "errors": "Book does not exist",
    "status": 400
}
```

Trying to register an already existing email produces:

```
{
    "errors": "Username or email already exists",
    "status": 400
}
```

And attempting to submit a review without borrowing the book first returns:

```
{
    "errors": "User has not borrowed this book",
    "status": 400
}
```

These validation tests helped confirm that both the API logic and the PostgreSQL constraints were behaving consistently even under invalid or unexpected requests.

8 Execution Report

8.1 Task Distribution

The work is divided between the two members, Clara and Gabriel, aiming to maintain a balance in effort.

Clara Pérez Alonso was primarily responsible for creating the REST API, including the sixteen endpoints, authentication logic, error handling, and transaction management. She also performed the testing in Postman.

Gabriel Furtado was primarily responsible for the synthetic data generation script, producing all the CSV files used to populate the database. He was also responsible for the video demonstration, the data loading script, and the database initialization.

We both collaborated on the database design and schema, resolving issues that arose during development and were identified in the first delivery, and reviewing each other's work at the end to ensure consistency and proper functionality. The project report was also written jointly.

9 Design Decisions

Throughout the development of the project, several design decisions had to be made regarding the database structure, API behavior, transaction handling, and even the testing strategy itself. Some choices were obvious from the beginning, while others only appeared after running into limitations or unexpected bugs during implementation.

In many cases, the final solution was not necessarily the first one we thought about. A few parts of the system went through multiple iterations before becoming stable enough to work reliably with larger datasets and concurrent requests.

Single Users Table with Role Subtables

One of the earliest decisions concerned how to represent user roles inside the database. Instead of creating completely separate tables for administrators, librarians, and readers, each duplicating fields like username, email, and password, the system uses a single users table containing all shared information.

The role hierarchy is then implemented through three smaller tables:administrator, librarian, and readers. Each one only stores a foreign key pointing back to the corresponding user in users.

This approach significantly reduces redundancy and keeps authentication much simpler. Every login query always checks the same table first, regardless of the user's role. After authentication succeeds, the API verifies permissions by checking the corresponding role table.

Another advantage is maintainability. If additional common user attributes needed to be added later, they would only need to exist in one place instead of being duplicated across multiple tables.

Intermediate Tables for Many-to-Many Relationships

Instead of storing authors and genres directly inside the `book` table as comma-separated values, the system uses the intermediate tables `book_author` and `book_genre`.

This approach avoids redundancy and makes SQL queries much simpler and more scalable. Operations such as retrieving all books written by a specific author or listing all books in a genre can be performed efficiently through SQL joins.

Using intermediate tables also simplified the insertion process implemented in `api.py`, where books are created first and linked to authors and genres afterwards.

Available Copies Calculated Dynamically

The system does not store the number of available copies as a persistent field inside the database. Instead, availability is calculated dynamically by subtracting the number of active loans from the total number of copies registered for the book.

For example:

```
b.num_copies - COUNT(DISTINCT(l.loan_id)) AS available_copies
```

At first, storing available copies directly seemed simpler. However, that approach would require updating the value every time a borrow or return operation occurred. Under concurrent requests, this can easily lead to synchronization problems or inconsistent counts if transactions fail midway.

It is also important to emphasize the use of `DISTINCT` because, by design, the database stores multiple rows for the same book in `book_genre`, one for each genre associated with that book. Since the query joins `book`, `book_genre`, and `loan`, the same loan can appear multiple times in the joined result, once per genre. Using `COUNT(DISTINCT l.loan_id)` ensures that each loan is counted only once, preventing SQL from counting duplicated joined rows and avoiding an incorrect reduction of the available copies.

Calculating availability directly from the loan table avoids that entire category of problems and guarantees that the value always reflects the actual state of the database.

NULL Return Date for Active Loans

Loan management was designed around a relatively simple idea: active loans are represented by rows whose `return_date` is still `NULL`.

When a reader borrows a book, a new loan record is inserted without a return date. Once the book is returned, the existing record is updated with the current timestamp instead of being deleted.

This decision preserves the complete borrowing history of the library, which is essential for endpoints such as:

- `/loaned_book`
- `/TopLoanedBooks/{N}`
- `/TopLoaners/{N}`
- `/report/TopLoanedGenres12Months/{N}`

It also simplifies queries involving active loans because they can be filtered easily using:

```
WHERE return_date IS NULL
```

Because this condition appears very frequently throughout the API, partial indexes were created specifically for active loan queries in `create_db.sql`.

Median Rating Computed in SQL

The endpoint `/available_books_by_genre` returns a rating value for each book based on user reviews. Instead of calculating this value in Python after fetching all ratings, the API computes the median directly inside PostgreSQL using:

```
PERCENTILE_CONT(0.5)
WITHIN GROUP (ORDER BY rating)
```

This keeps all aggregation logic inside the database engine, which was one of the requirements stated in the project specification.

It also avoids transferring unnecessary review data into Python memory only to compute a single statistical value afterwards. Even though the current dataset is not extremely large, keeping heavy processing inside SQL scales much better.

Concurrency Control with SELECT FOR UPDATE

Concurrency handling became especially important for the borrow endpoint. Without synchronization, two users could potentially borrow the last available copy of the same book simultaneously.

To prevent this race condition, the borrow transaction locks the relevant rows using PostgreSQL's `SELECT FOR UPDATE` instruction before validating availability.

Once a transaction locks the selected book row, any concurrent transaction attempting to borrow the same book must wait until the first one commits or rolls back.

This solution turned out to be one of the most important technical decisions in the entire project because it guarantees database consistency even under concurrent requests running with Flask threaded mode enabled.

Additional Add Author Endpoint

The original specification did not explicitly include an endpoint for registering authors. However, during development we quickly realized that the `/book` endpoint required authors to already exist in the database before a book could be inserted.

Without an author creation endpoint, new authors would have to be inserted manually through SQL, which would make the API incomplete from a practical point of view.

Because of that, an additional administrator-only endpoint was implemented:

```
/sgdproj/author
```

This endpoint allows administrators to register authors directly through the API while preserving the same validation and authentication workflow used by the rest of the system.

Review Update Logic

The project specification states that readers cannot create multiple independent reviews for the same book. Instead, if a reader submits another review for a previously reviewed book, the existing review must be updated.

To implement this behavior, the endpoint first checks whether a review already exists for the corresponding reader-book combination:

```
SELECT review_id
FROM review
WHERE readers_users_user_id = %s
AND book_isbn = %s
```

If a matching review exists, the system performs an `UPDATE`; otherwise, it inserts a new review row.

Although PostgreSQL supports database-level UPSERT operations, this logic was implemented explicitly inside `api.py` because it makes the workflow easier to understand and debug during development.

Postman-Based Validation Strategy

From the beginning of the project, we maintained a structured Postman collection instead of testing endpoints manually one by one. Each time a new endpoint was implemented or modified, it was immediately tested against the predefined requests in the collection.

This approach helped us catch validation issues, permission errors, and transaction problems much faster than manual testing would have allowed. The collection eventually grew to cover not only successful requests but also intentional error cases, which proved especially useful during the final stages of development when the system needed to be validated as a whole rather than endpoint by endpoint.

Authentication in Reader Operations

Although the specification suggests using user_id for operations such as borrowing and returning books, we decided to use username and password instead. This keeps authentication consistent across all endpoints and avoids exposing internal database IDs in client requests.

10 AI Disclosure

During the development of this project, AI-based tools such as ChatGPT, Claude, and Gemini were used in a limited and controlled manner. Specifically, AI was consulted occasionally to help translate or interpret PostgreSQL or Python error messages and to suggest fixes for minor syntax issues in SQL queries or Flask path definitions. In some cases, AI also helped rephrase or clarify warning messages and log output. However, no AI tools were used to generate the core logic, database schema, API endpoint implementations, transaction management code, or synthetic data generation scripts. All design decisions, the complete implementation of the 16 required endpoints, concurrency control, authentication and authorization logic, and business rule validation were written entirely by Clara and Gabriel.

11 Conclusions

This project allowed us to apply in practice many of the concepts studied throughout the course, especially relational database design, transaction management, concurrency control, and REST API development.

Although the system initially seemed relatively simple, the implementation quickly became more challenging as new business rules, validations, and concurrent operations were introduced. Managing consistency during borrowing operations, handling user permissions correctly, and maintaining reliable transaction behavior became some of the most important parts of the project.

One of the most valuable aspects of the development process was working with a larger synthetic dataset. Testing the API with thousands of loans, books, and users helped expose issues that would not appear with small manual datasets, particularly in validation logic, SQL queries, and transaction handling.

The project also reinforced the importance of careful database design. Decisions regarding constraints, foreign keys, indexes, and relationship modeling directly affected the behavior and complexity of the API implementation.

The API supports all required operations, enforces role-based permissions, maintains database consistency during concurrent requests, and handles invalid operations through controlled error responses.

Overall, the project provided valuable practical experience in building a complete database-driven application using PostgreSQL, Flask, and Python, while also improving our understanding of transaction management, API design, and system validation.